Transition Is Not Viability: Action-Conditioned World Models for LLM Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

World-model supervision for LLM agents typically trains models to predict the next state under an action. We argue that this target is behaviorally incomplete: in many planning environments, an action can produce a locally valid next state that is already unrecoverable. We formalize this missing capability as *action-conditioned viability prediction*: estimating whether the successor state induced by a proposed action still admits a successful continuation. We introduce **SAVE** (Sibling-Action Viability Estimation), a world-modeling recipe that labels multiple candidate actions considered at the same decision step with both their successor states and solver-derived viability. This sibling-action supervision enables same-state contrastive learning: the model must distinguish actions that look locally similar but differ in whether they preserve recoverability. At inference, SAVE is used through a calibrated viability-constrained policy, which filters candidate actions by predicted successor viability and terminates only under candidate-set exhaustion. Across Sudoku, Pentomino tiling, and Hidato, we evaluate whether viability emerges from transition modeling, whether it separates viable from doomed sibling actions, and whether it improves candidate selection under matched inference budgets. Our results show that transition accuracy and viability prediction can dissociate, that progress/value scoring fails on deceptive surface-valid actions, and that calibrated viability filtering reduces dead-end entry while enabling conservative rollout truncation.

1 Introduction

A world model for an agent is usually understood as a predictor of what will happen after the agent acts. For LLM agents, recent work has turned this idea into supervised fine-tuning objectives that train the model to predict the next textual or symbolic state during self-play. Such transition prediction is useful, but it is not sufficient for decision making. In many planning environments, an action can be locally legal and its next state can be predicted perfectly, while the resulting state is already unrecoverable: no continuation can still reach the goal.

We study this missing capability as *viability prediction*. Given a state s and a proposed action a, the target is not only the successor state $T(s, a)$, but whether that successor remains recoverable: $v(T(s, a)) = 1$ if some continuation can still succeed, and 0 otherwise. This action-conditioned formulation is crucial. A state-level solvability tag can say whether the current state is dead; it cannot compare two candidate actions from the same state, one of which preserves a solution path and one of which silently destroys it.

This distinction creates a predictive gap for LLM-agent world models. Transition accuracy may be high even when the model cannot identify which surface-valid actions enter dead ends. Conversely, progress or value scores can prefer actions that look locally promising but eliminate all successful

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

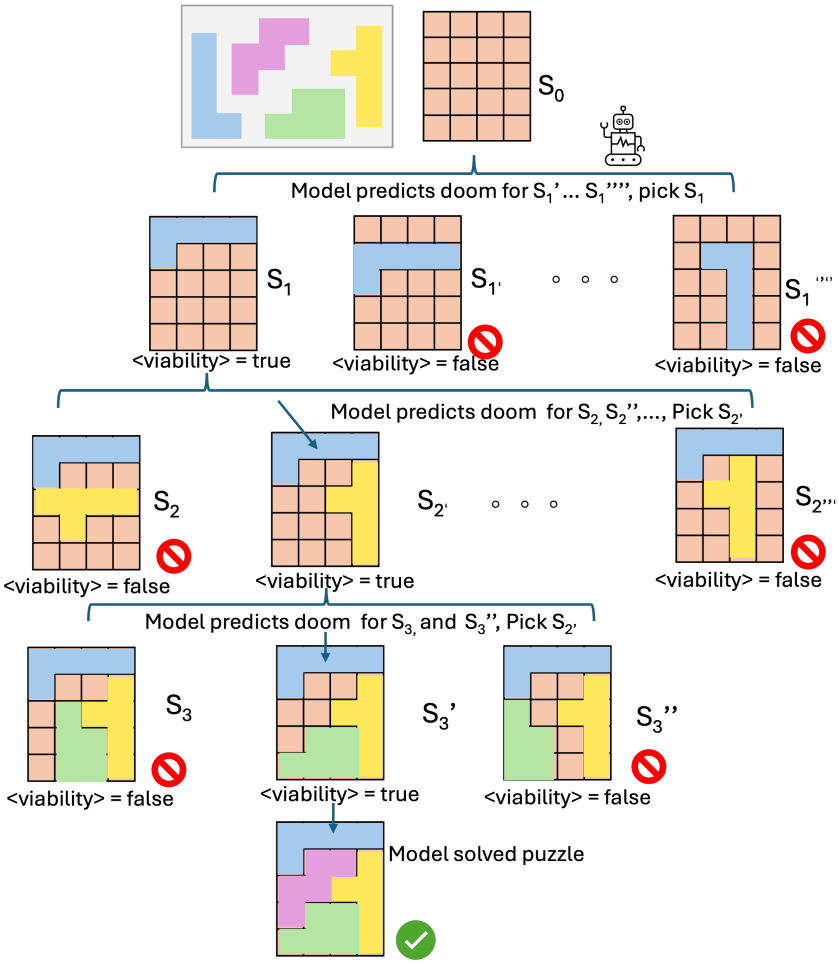

Figure 1: **Viability prediction in Pentomino tiling.** A representative rollout trajectory illustrating the central concept of this work. From an empty 5×4 board with five pentominoes to place (top), the agent must select a sequence of placements that fully tiles the board. At each decision step, three candidate placements are shown; red prohibition marks indicate *doomed* successors—boards where, despite the placement being locally legal and visually compact, no continuation can complete the tiling. Only one branch (highlighted arrow) preserves the existence of a winning continuation; selecting any other branch silently commits the rollout to eventual failure many steps before the failure becomes locally observable. SAVE supervises the agent to predict, per candidate, whether the resulting state remains recoverable—the *viability* of $T(s_t, a)$—rather than only what the next state will be.

continuations. We therefore ask three questions: Does viability prediction emerge automatically from transition modeling? Can an LLM world model learn to rank viable against doomed sibling actions from the same state? And can this signal improve action selection without giving the agent more test-time compute?

We introduce **SAVE** (Sibling-Action Viability Estimation). At each decision state, we collect sibling actions: candidate actions considered by the policy before commitment. Each sibling is transitioned through the environment and labeled by a solver for successor-state viability. SAVE trains a single action-conditioned world model to predict both the next state and whether that next state remains viable, with an auxiliary same-state contrastive loss on mixed sibling sets.

At inference, we use the learned viability model through a calibrated viability-constrained policy. The policy samples K candidate actions, scores each by predicted successor viability, filters candidates above a validation-calibrated threshold, and then lets the base policy choose among the surviving actions. If no sampled candidate exceeds a lower fallback threshold, the rollout is terminated

under the fixed candidate budget. This termination rule should be read as candidate-set exhaustion, not as a proof that the full state is unsolvable.

Our experiments on Sudoku, Pentomino tiling, and Hidato evaluate viability as a world-model capability rather than as an auxiliary tag. We test whether transition and viability prediction dissociate across model scales, whether SAVE separates viable from doomed sibling actions, whether viability outperforms learned progress/value scoring on deceptive candidates, and whether calibrated filtering reduces dead-end entry under matched inference budgets.

Our contributions are:

- **Action-conditioned viability as a world-model target.** We formalize successor-state viability, $v(T(s, a))$, as a planning-relevant prediction target distinct from next-state prediction and state-level solvability classification.
- **Sibling-action supervision for viability learning.** We introduce SAVE, which labels multiple candidate actions considered at the same state and trains a world model with per-action viability supervision and same-state contrastive ranking on mixed sibling sets.
- **A deceptive-candidate evaluation for recoverability.** We construct cases where local progress or learned utility scores prefer surface-valid but doomed actions, isolating whether a method predicts recoverability rather than progress.
- **Calibrated viability-constrained inference.** We introduce a candidate-set filtering and termination operator calibrated on validation data, and evaluate its effect on dead-end entry, false termination, Pass@1, and rollout compute under matched inference budgets.

2 Method

We present **SAVE** (**S**ibling-**A**ction **V**iability **E**stimation), an action-conditioned world-modeling framework that supervises LLM agents to predict not only the consequence of an action but whether that consequence remains recoverable. SAVE has three components: an action-conditioned formulation that estimates successor-state viability per candidate action (§2.1–§2.2); a boundary-aware contrastive training scheme on *sibling actions* sampled at the same decision step (§2.3); and an inference-time operator, the *Calibrated Viability-Constrained Policy* (CVCP), with a dead-end safety guarantee (§2.4). Section 2.5 integrates these into a deployment algorithm; Section 2.6 extends CVCP to early termination of doomed rollouts.

2.1 Setup

We model an LLM agent acting in a deterministic environment with state space $\mathcal{S}$, action space $\mathcal{A}$, transition $T : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$, and goal set $\mathcal{S}_g \subseteq \mathcal{S}$. The *viability function* $v : \mathcal{S} \to \{0, 1\}$ is defined by

$$v(s) \;=\; 1\big[\,\exists\,\pi \,:\, \Pr_\pi\big[\,s_{0:H} \cap \mathcal{S}_g \neq \emptyset \,\big|\, s_0 = s\,\big] > 0\,\big]\,. \tag{1}$$

A state with $v(s) = 0$ is a *dead end*: no continuation reaches the goal. We assume access to an oracle solver that computes v exactly; this is realized in our three environments—Sudoku, Pentomino tiling, and Hidato (a connected-path number-placement puzzle on a grid)—by constraint propagation and bounded-depth search.[1] We write $V^\pi(s)$ for the expected task return of π from s.

SAVE targets two capabilities of an internalized world model that prior LLM-agent SFT pipelines [Chen et al., 2025, Wang et al., 2025] address only partially: *transition prediction* (estimating $T(s, a)$) and *viability prediction* (estimating $v(T(s, a))$). Section 3 provides empirical evidence that the two are *dissociable*—following independent scaling curves with respect to model capacity—and motivates supervising them jointly.

2.2 Action-Conditioned Viability World Model

We jointly parameterize transition prediction, action-conditioned viability prediction, and an auxiliary state-conditioned viability prediction in a single autoregressive language model f_ϕ. Given a

[1] In Hidato, the agent fills cells with consecutive integers $1, 2, \ldots, N$ such that consecutive numbers occupy adjacent cells; viability reduces to the existence of a Hamiltonian path completion from the current partial assignment, decidable by backtracking search on instance sizes used here.

rendered state s and a proposed action a, the model emits a structured sequence containing three tagged fields: a predicted successor state $\hat{s}'$, a per-action viability label $\hat{y} \in \{\texttt{true}, \texttt{false}\}$ predicting $v(T(s,a))$, and a state-conditioned viability label $\hat{u} \in \{\texttt{true}, \texttt{false}\}$ predicting $v(s)$. The exact output schema (XML-style tags `next_state`, `viability`, and `state_viable`) is given in Appendix A. The corresponding probabilities are

$$\hat{v}_\phi(s,a) := P_{f_\phi}\big(\hat{y} = \texttt{true} \,\big|\, s, a\big)\,, \qquad \hat{u}_\phi(s) := P_{f_\phi}\big(\hat{u} = \texttt{true} \,\big|\, s\big)\,. \tag{2}$$

The action-conditioned head $\hat{v}_\phi$ drives both training supervision (§2.3) and inference-time filtering (§2.4). The state-conditioned head $\hat{u}_\phi$ is trained jointly but used only for evaluation: it provides the diagnostic axis for the capacity dissociation analysis in §3 and is *not* consulted by the inference algorithm (Algorithm 2).

The action-conditioned input pair (s,a), rather than s alone, distinguishes SAVE from state-conditioned viability tags in prior work [Chen et al., 2025, Wang et al., 2025], where the model emits a single viability flag for the current state alongside its own chosen action—a setup that conflates the predictive target with the policy's preferred action and prevents per-candidate evaluation. The action-conditioned formulation enables (i) per-candidate filtering at inference time (§2.4); (ii) sibling-action contrastive supervision (§2.3); and (iii) the dead-end safety guarantee in Proposition 1.

2.3 Sibling-Action Supervision

Sibling sets. For each visited state s_t, we form a *sibling set* $A_t = \{a_t^{(1)}, \ldots, a_t^{(K)}\}$ of K candidate actions considered at the same decision step. Each sibling is independently transitioned and labeled by the oracle solver:

$$s_{t+1}^{(i)} = T\Big(s_t, a_t^{(i)}\Big)\,, \qquad y_t^{(i)} = v\Big(s_{t+1}^{(i)}\Big) \in \{0,1\}. \tag{3}$$

We also record the state-level label $u_t = v(s_t) \in \{0,1\}$ for training the auxiliary head $\hat{u}_\phi$. Each training instance is the tuple $\Big(s_t,\, A_t,\, \{(s_{t+1}^{(i)}, y_t^{(i)})\}_i,\, u_t\Big)$. Unlike post-hoc counterfactual generation [Chu et al., 2025], sibling actions are *contemporaneous*: they arise from the policy's own sampling distribution at the original decision step, before any commitment, and incur no additional inference-time generation cost beyond candidates the policy would already sample.

Boundary-aware sibling construction. Naive sampling of K candidates from π_θ produces unbalanced sibling sets: easy states yield all-viable groups, near-dead-end states yield all-doomed groups, and the contrastive pairs needed below are concentrated on a small fraction of *boundary* states. We address this by (i) drawing siblings from a mixture of on-policy, exploratory, and solver-derived sources to ensure both class-coverage and hard negatives, and (ii) up-weighting states with high *class-mixedness* $\mathrm{mix}(s) = \min(\#\text{viable}, \#\text{doomed})/\#\text{legal}$. The full sampling protocol is described in Appendix B.[2]

Objective. Let f_ϕ produce per-action logit $\ell^{(i)}$ with $\hat{v}_\phi(s_t, a_t^{(i)}) = \sigma(\ell^{(i)})$. The training loss combines four terms:

$$\mathcal{L}_{\text{trans}} = -\sum_{i=1}^{K} \log P_{f_\phi}\Big(s_{t+1}^{(i)} \,\big|\, s_t, a_t^{(i)}\Big)\,, \tag{4}$$

$$\mathcal{L}_{\text{viab}} = -\sum_{i=1}^{K} \Big[\, y_t^{(i)} \log \hat{v}_\phi(s_t, a_t^{(i)}) + (1 - y_t^{(i)}) \log\Big(1 - \hat{v}_\phi(s_t, a_t^{(i)})\Big) \,\Big]\,, \tag{5}$$

$$\mathcal{L}_{\text{rank}} = -\, 1_{\mathcal{P}_t \neq \emptyset} \sum_{(i,j)\in\mathcal{P}_t} \log \sigma\Big(\ell^{(i)} - \ell^{(j)}\Big)\,, \tag{6}$$

$$\mathcal{L}_{\text{state}} = -\,[\, u_t \log \hat{u}_\phi(s_t) + (1 - u_t) \log(1 - \hat{u}_\phi(s_t)) \,]\,, \tag{7}$$

[2] For environments with small enumerable action spaces (Sudoku, Pentomino, Hidato), the *perturbed solver-path* source is constructed by local edits to a viable trajectory that preserve single-step format validity but introduce downstream constraint violations (detailed per-environment in Appendix C).

Algorithm 1 SAVE training step (per minibatch).

Require: sibling-set minibatch $\{(s_t^{(b)}, A_t^{(b)}, \{(s_{t+1}^{(b,i)}, y_t^{(b,i)})\}_{i=1}^{K}, u_t^{(b)})\}_{b=1}^{B}$, model f_ϕ, loss weights (λ, η, μ).

Forward pass $f_\phi(s_t^{(b)}, A_t^{(b)})$ for each b, producing per-action logits $\ell^{(b,i)}$, predicted successor states $\hat{s}_{t+1}^{(b,i)}$, and state-conditioned prediction $\hat{u}_\phi(s_t^{(b)})$.
Compute $\mathcal{L}_{\text{trans}}$ from Eq. 4 using $\hat{s}_{t+1}^{(b,i)}$ vs. $s_{t+1}^{(b,i)}$.
Compute $\mathcal{L}_{\text{viab}}$ from Eq. 5 using labels $y_t^{(b,i)}$.
Initialize $\mathcal{L}_{\text{rank}} \leftarrow 0$.
for $b = 1$ to B **do**
  Form viable–doomed pair set $\mathcal{P}_t^{(b)} = \{(i,j) : y_t^{(b,i)} = 1,\ y_t^{(b,j)} = 0\}$.
  if $\mathcal{P}_t^{(b)} \neq \emptyset$ **then**
    $\mathcal{L}_{\text{rank}} \mathrel{+}= -\sum_{(i,j)\in\mathcal{P}_t^{(b)}} \log \sigma\big(\ell^{(b,i)} - \ell^{(b,j)}\big)$.
  end if
end for
Compute $\mathcal{L}_{\text{state}}$ from Eq. 7 using labels $u_t^{(b)}$.
Total loss: $\mathcal{L} \leftarrow \mathcal{L}_{\text{trans}} + \lambda\mathcal{L}_{\text{viab}} + \eta\mathcal{L}_{\text{rank}} + \mu\mathcal{L}_{\text{state}}$.
Backpropagate through $\mathcal{L}$ and update ϕ via AdamW.

where $\mathcal{P}_t = \{(i,j) : y_t^{(i)} = 1,\ y_t^{(j)} = 0\}$ is the set of viable–doomed pairs within the sibling set. The total objective is

$$\mathcal{L} = \mathcal{L}_{\text{trans}} + \lambda\mathcal{L}_{\text{viab}} + \eta\mathcal{L}_{\text{rank}} + \mu\mathcal{L}_{\text{state}}, \tag{8}$$

with weights (λ, η, μ) given in Appendix D.

The ranking term (6) is the key component for distinguishing SAVE from a marginal viability classifier. Pure cross-entropy on viability labels can attain high accuracy by exploiting state-level priors (some boards are simply harder than others) without learning the action-dependent structure that distinguishes a viable continuation from a doomed one *at the same state*. The pairwise objective on $\mathcal{P}_t$, activated only on *mixed* sibling sets where both classes co-occur, enforces precisely this distinction.

Algorithm 1 specifies the training step explicitly, making the conditional activation of $\mathcal{L}_{\text{rank}}$ on mixed sibling groups visible.

2.4 Calibrated Viability-Constrained Policy (CVCP)

Given a trained $\hat{v}_\phi$ and threshold $\tau \in (0,1)$, define

$$\tilde{\pi}_\tau(a \mid s) = \frac{\pi_\theta(a \mid s) \cdot 1[\hat{v}_\phi(s,a) \geq \tau]}{Z_\tau(s)}, \qquad Z_\tau(s) = \sum_{a' \in \mathcal{A}} \pi_\theta(a' \mid s) \cdot 1[\hat{v}_\phi(s,a') \geq \tau], \tag{9}$$

defined when $Z_\tau(s) > 0$ (the empty-kept-set fallback is treated in §2.5). At decision time, the agent samples K candidates from π_θ, scores each with $\hat{v}_\phi$, retains those passing τ, and selects among the survivors according to the original policy's preference. CVCP is parameter-free at inference: τ is fixed by validation calibration and π_θ is unchanged.

Proposition 1 (Dead-end safety under calibrated viability filtering)**.** *Fix threshold τ and let $C_\tau(s) = \{a : \hat{v}_\phi(s,a) \geq \tau\}$. Suppose $\hat{v}_\phi$ has* precision-on-True *at level τ under the base-policy occupancy d^{π_θ}:*

$$\Pr_{\substack{s \sim d^{\pi_\theta} \\ a \sim \pi_\theta(\cdot \mid s)}} \big[\, v(T(s,a)) = 0 \,\big|\, \hat{v}_\phi(s,a) \geq \tau \,\big] \ \leq\ \varepsilon. \tag{10}$$

For every state s in the support of d^{π_θ} with $C_\tau(s) \neq \emptyset$:

(i) ***One-step dead-end safety****:* $\Pr_{a \sim \tilde{\pi}_\tau(\cdot \mid s)}[v(T(s,a)) = 0] \leq \varepsilon$.

(ii) ***Strict relative improvement****: let $\beta(s) := \Pr_{a \sim \pi_\theta}[v(T(s,a)) = 0]$ be the base-policy one-step dead-end rate. Then*

$$\Pr_{a \sim \pi_\theta}[v(T(s,a)) = 0] \ -\ \Pr_{a \sim \tilde{\pi}_\tau}[v(T(s,a)) = 0] \ \geq\ \beta(s) - \varepsilon,$$

with strict improvement whenever $\beta(s) > \varepsilon$.

Algorithm 2 SAVE inference: CVCP with low-confidence fallback termination.

Require: state s_t, base policy π_θ, viability model $\hat{v}_\phi$, keep threshold τ_{keep}, fallback threshold τ_{fb} with $\tau_{\text{fb}} < \tau_{\text{keep}}$, candidate count K.

1: Sample K siblings: $A_t = \{a^{(i)}\}_{i=1}^{K} \sim \pi_\theta(\cdot \mid s_t)$.
2: Compute viability scores: $p_i \leftarrow \hat{v}_\phi(s_t, a^{(i)})$ for $i = 1, \ldots, K$.
3: Form kept set: $C_\tau \leftarrow \{a^{(i)} : p_i \geq \tau_{\text{keep}}\}$.
4: **if** $C_\tau \neq \emptyset$ **then**
5: **return** $\arg\max_{a \in C_\tau} \log \pi_\theta(a \mid s_t)$. ▷ CVCP main path; Prop. 1 applies.
6: **else**
7: **if** $\max_i p_i < \tau_{\text{fb}}$ **then**
8: **return** TERMINATE. ▷ All siblings score below fallback threshold; rollout treated as a sibling-set dead-end.
9: **else**
10: **return** $\arg\max_i p_i$. ▷ Fallback execution; safety bound does not apply.
11: **end if**
12: **end if**

Proof. **(i)** The renormalized policy $\tilde{\pi}_\tau(\cdot \mid s)$ is supported on $C_\tau(s)$, with mass proportional to $\pi_\theta(a \mid s)$. Therefore

$$\Pr_{a \sim \tilde{\pi}_\tau}[v(T(s,a)) = 0] \;=\; \Pr_{a \sim \pi_\theta}\big[v(T(s,a)) = 0 \,\big|\, \hat{v}_\phi(s,a) \geq \tau\big] \;\leq\; \varepsilon,$$

where the equality follows from renormalization preserving conditional probabilities, and the inequality is the precision-on-True assumption (10).

(ii) By definition $\beta(s) = \Pr_{a \sim \pi_\theta}[v(T(s,a)) = 0]$. By (i), $\Pr_{a \sim \tilde{\pi}_\tau}[v(T(s,a)) = 0] \leq \varepsilon$. Subtracting yields the stated lower bound on improvement. Strict inequality holds when $\beta(s) > \varepsilon$. □

Reading the bound. ε in (10) is directly measurable from a validation threshold sweep (Section 3, Table **??**). Choosing τ navigates the precision–recall trade-off: large τ reduces ε (safer) but shrinks $C_\tau(s)$, increasing the rate of empty-kept-set fallbacks (§2.6). Two caveats are worth noting. First, the assumption holds on the base-policy distribution; we calibrate τ on a held-out validation set drawn from π_θ (distinct from the boundary-aware training mixture) and report the realized deployment-time false-viability rate to verify the assumption empirically (Section 3, Figure **??**). Second, Proposition 1 bounds dead-end *entry* rate, not value: in deterministic goal-reaching tasks where dead ends carry zero return, this implies weak value preservation under additional regularity, but we do not claim universal value improvement. The safety bound is the operational property required for CVCP to be deployable as a rollout-truncation gate during agentic RL.

2.5 Algorithm

Algorithm 2 integrates the components at inference time. Given the simplicity of the resulting procedure—two thresholds, no auxiliary heads consulted at inference—we present it together with its empty-kept-set behavior.

Two thresholds. τ_{keep} governs the per-candidate filter and is calibrated to satisfy the precision-on-True target ε of Proposition 1. The fallback threshold τ_{fb} is a strictly lower threshold ($\tau_{\text{fb}} < \tau_{\text{keep}}$) applied only when $C_\tau = \emptyset$: it determines whether the empty kept set reflects an unrecoverable state (terminate) or merely an unfortunate sample of candidates (fall back to the highest-scoring action). Concretely, τ_{fb} is calibrated on the validation set so that the conditional event $\{\max_i p_i < \tau_{\text{fb}}\}$ has precision-on-False (true dead-end given termination) at least $1 - \varepsilon_{\text{fb}}$, with ε_{fb} stricter than ε because terminating an entire rollout is more costly than executing a single sub-threshold action (Appendix D). Selected values per environment are reported in Section 3.

Fallback safety. When the fallback execution branch (Algorithm 2, line 9) is triggered, the dead-end safety bound of Proposition 1 does not apply for that step: the executed action has $\hat{v}_\phi(s,a) < \tau_{\text{keep}}$ by construction. We accept this as an asymmetric trade-off: discarding a state where the agent's best available evidence still places non-trivial probability on viability would cost more in

expected return than executing the most-viable-looking action. Section 3 reports fallback-execution rates and the conditional dead-end entry rates within fallback steps.

2.6 Application: Mid-Rollout Truncation for Compute Savings

The TERMINATE branch of Algorithm 2 (line 7) embodies SAVE's central operational application. When the viability head reports that all K sampled candidates score below τ_{fb}, the rollout is treated as having entered a dead end and is aborted. Compared to running the rollout to its natural length, mid-rollout truncation saves the inference cost of every remaining step. We report token-level compute savings, the true-positive rate of triggered terminations, and the false-termination rate (recoverable states incorrectly aborted) in Section 3.

The semantic distinction underlying this design—between a state that admits no successful continuation and a state where the policy's K-sample happened to be unfortunate—is not directly observable from $\{p_i\}_{i=1}^{K}$ alone. The threshold τ_{fb} trades off these two interpretations: setting τ_{fb} very low guarantees that triggered terminations correspond to genuinely unrecoverable states (low false-termination rate) but rarely fires (low compute savings); setting it close to τ_{keep} fires often (high compute savings) but conflates the two cases. The calibration protocol in Appendix D selects τ_{fb} to bound the false-termination rate at $\varepsilon_{\text{fb}} = 0.05$ on the validation set.

3 Experiments

We evaluate SAVE along two axes. First, we ask whether successor-state viability is a distinct world-modeling capability rather than a by-product of transition prediction or progress scoring. Second, we ask whether the learned viability signal changes agent behavior under matched inference budgets. The experiments are organized around five questions:

1. Does viability prediction co-emerge with transition prediction as model scale increases?
2. Does SAVE learn action-conditioned viability, i.e., can it rank viable and doomed actions from the same state?
3. Is viability distinct from progress or value scoring on surface-valid but deceptive actions?
4. Does calibrated viability filtering reduce dead-end entry during online rollouts without simply spending more test-time compute?
5. Can candidate-exhaustion termination save rollout compute while controlling false termination?

Unless otherwise stated, all reported numbers are computed on held-out test instances. Validation data is used only for temperature scaling, threshold selection, and early stopping.

3.1 Experimental Setup

Environments. We use three deterministic puzzle environments in which local validity and global recoverability can diverge. In **Sudoku**, an action may satisfy the current row, column, and box constraints while making the partial board unsolvable. In **Pentomino tiling**, a placement may cover area and look locally compact while leaving an empty region that cannot be tiled by the remaining pieces. In **Hidato**, a number placement may satisfy local adjacency while preventing any completion of the required consecutive path. For every reached state s, an oracle solver computes the viability label $v(s)$, indicating whether some legal continuation can still reach a goal state.

Models and training data. We fine-tune Qwen2.5-Instruct models at [TBD] parameter sizes. For each environment, we construct sibling-action datasets following the protocol in Section 2.3. Each training example contains a state s_t, a set of K candidate actions sampled from the base policy, the environment successor $T(s_t, a^{(i)})$ for each candidate, and the solver-derived successor viability $v(T(s_t, a^{(i)}))$. Train, validation, and test splits are constructed at the trajectory level to avoid leakage between near-identical states.

[Placeholder for transition accuracy and viability AUC scaling curves.]

Figure 2: Scaling behavior of transition prediction and successor-viability prediction. Each curve is evaluated on the same held-out state-action distribution. A separation between the curves indicates that predicting what happens next is not sufficient to predict whether the next state remains recoverable.

Calibration. SAVE outputs a probability $\hat{v}_\phi(s, a)$ that the successor induced by action a remains viable. We calibrate this probability on validation data using [temperature scaling / isotonic regression / TBD]. Two thresholds are selected on the validation split. The keep threshold τ_{keep} is chosen to obtain high precision among retained actions:

$$\Pr[v(T(s, a)) = 1 \mid \hat{v}_\phi(s, a) \geq \tau_{\text{keep}}] .$$

The fallback threshold τ_{fb} is chosen for conservative candidate-exhaustion termination:

$$\Pr[v(T(s, a)) = 0 \mid \hat{v}_\phi(s, a) < \tau_{\text{fb}}] .$$

We keep these thresholds fixed for all test-time evaluations.

Baselines. All multi-candidate methods use the same candidate set, the same sampling temperature, and the same candidate budget K. This controls for the benefit of seeing more actions at inference time.

- **Policy top-1**: executes the highest-probability action under the base policy.
- **Best-of-K policy**: samples K actions and executes the one with highest policy probability. This controls for candidate generation without any world-model filtering.
- **Local progress heuristic**: ranks candidates by a hand-designed progress function, such as filled cells in Sudoku, placed area in Pentomino, or path coverage in Hidato, with environment-specific penalties for immediate constraint violations.
- **Prompted score-only**: prompts the same base model to assign each candidate outcome a scalar utility score, then executes the highest-scoring action. This represents an inference-time outcome-scoring baseline.
- **Learned progress-score**: uses the same backbone, input format, SFT budget, and candidate budget as SAVE, but replaces the viability target with a continuous progress or residual-difficulty target $q(s, a)$. At inference, the action with the highest predicted score $\hat{q}_\psi(s, a)$ is executed. This baseline matches SAVE in training quality and inference cost; the difference is the supervision signal.
- **SAVE**: filters candidates by predicted successor viability and then lets the base policy choose among the surviving actions, as in Algorithm 2.
- **Oracle viability**: uses the solver label $v(T(s, a))$ to filter candidates. This is not deployable, but gives an upper bound on the value of perfect viability prediction.

Metrics. We report both prediction metrics and behavioral metrics. Prediction quality is measured by ROC-AUC, PR-AUC on the doomed class, Brier score, expected calibration error (ECE), and same-state pairwise ranking accuracy. Rollout quality is measured by Pass@1, dead-end entry rate, first dead-end step, non-viable selected-action rate, false pruning rate, tokens per episode, and net compute. Net compute includes the cost of per-candidate model evaluation, not only the length of the final executed trajectory.

3.2 Transition Accuracy Does Not Imply Viability Prediction

We first test whether viability prediction emerges automatically from transition modeling. For each model size and environment, we evaluate the same held-out state-action pairs on two targets: predicting the successor state $T(s, a)$ and predicting whether that successor remains viable, $v(T(s, a))$. Transition prediction is measured by [exact match / structured accuracy / token-level accuracy], while viability prediction is measured by ROC-AUC.

This experiment is diagnostic rather than deployment-oriented. If transition accuracy becomes high while viability AUC remains near chance, then standard world-model evaluation would overstate

Table 1: Transition prediction and successor-viability prediction across model sizes. Transition accuracy measures whether the model predicts the immediate next state correctly. Viability AUC measures whether the model predicts whether that successor remains recoverable. The two quantities need not increase together.

Environment	Model	Transition Acc. ↑	Viability AUC ↑	Gap
4*Sudoku	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]
	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]
	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]
	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]
4*Pentomino	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]
	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]
	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]
	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]
4*Hidato	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]
	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]
	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]
	Qwen2.5-[TBD]	[TBD]	[TBD]	[TBD]

Table 2: Action-conditioned successor-viability prediction. Same-state pairwise accuracy is the key action-conditioned metric: chance performance indicates that the model does not distinguish viable and doomed siblings from the same state. Calibration metrics are reported because CVCP uses fixed thresholds at deployment.

Environment	ROC-AUC	PR-AUC (doomed)	Same-state pairwise	Brier	ECE	Prec@τ_{keep}
Sudoku	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Pentomino	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Hidato	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

the model's usefulness for planning: the model can describe the next state without recognizing that the next state has no successful continuation.

3.3 Action-Conditioned Viability Prediction

We next evaluate whether SAVE learns a genuinely action-conditioned signal. Marginal classification metrics alone are not enough: a model could perform well by memorizing which states are generally close to failure, while still failing to compare two actions from the same state. We therefore report same-state pairwise ranking accuracy. For each mixed sibling set containing at least one viable and one doomed action, we form pairs (a^+, a^-) such that

$$v(T(s, a^+)) = 1, \qquad v(T(s, a^-)) = 0,$$

and measure whether $\hat{v}_\phi(s, a^+) > \hat{v}_\phi(s, a^-)$.

For threshold-based deployment, the important property is not only whether the model separates classes on average, but whether high-confidence retained actions are actually viable. We therefore report retained-action precision at τ_{keep} and use the validation-selected threshold unchanged in the online rollout experiments below.

3.4 Deceptive Candidate Benchmark

The central comparison is between recoverability and progress. A score model can prefer actions that look locally good: they fill more cells, place more area, or extend more of a path. Viability asks a different question: whether the resulting state still has any successful continuation.

To isolate this distinction, we construct a deceptive candidate benchmark. Each example consists of one viable action a^+ and one doomed action a^- from the same state s:

$$v(T(s, a^+)) = 1, \qquad v(T(s, a^-)) = 0, \qquad \text{progress}(T(s, a^-)) \geq \text{progress}(T(s, a^+)). \quad (11)$$

[Placeholder for score distributions or reliability diagrams.]

Figure 3: Distribution and calibration of predicted successor viability $\hat{v}_\phi(s, a)$. The left panel separates viable and doomed successors; the right panel reports reliability after calibration.

Table 3: Deceptive candidate benchmark. Each pair contains a doomed action with equal or higher local progress than its viable sibling. The learned progress-score baseline matches SAVE in backbone, data budget, candidate budget, and inference cost; differences between the two isolate the effect of recoverability supervision.

Method	Sudoku	Pentomino	Hidato
Policy top-1	[TBD]	[TBD]	[TBD]
Best-of-K policy	[TBD]	[TBD]	[TBD]
Local progress heuristic	100.0	100.0	100.0
Prompted score-only	[TBD]	[TBD]	[TBD]
Learned progress-score	[TBD]	[TBD]	[TBD]
SAVE	**[TBD]**	**[TBD]**	**[TBD]**
Oracle viability	0.0	0.0	0.0

Thus the doomed action is at least as good as the viable action under the local progress heuristic. In Sudoku, such actions are locally legal digit placements that later force a contradiction. In Pentomino, they are placements that cover area while creating an untileable cavity. In Hidato, they are locally adjacent number placements that block any full path completion.

Each method chooses between a^+ and a^-. We report the non-viable selection rate: the fraction of examples on which the method selects a^-. Lower is better.

This benchmark is intentionally adversarial to progress scoring. A strong result here would show that SAVE is not merely learning a binary version of progress or utility. It is learning a constraint on whether the future solution set remains non-empty.

3.5 Online Rollout Under Matched Inference Budget

We then test whether viability prediction changes agent behavior during full rollouts. At each step, the policy samples K candidate actions. Each multi-candidate method receives the same candidates and is charged for its per-candidate evaluation. SAVE applies CVCP: candidates with $\hat{v}_\phi(s, a) \geq \tau_{\text{keep}}$ are retained, and the base policy chooses among the retained candidates. If no candidate is retained, SAVE uses the fallback behavior in Algorithm 2.

We report both task success and failure dynamics. Pass@1 alone can obscure the mechanism: a method may fail to solve the puzzle but still avoid irreversible mistakes for longer. We therefore measure dead-end entry rate, the first step at which a dead-end is entered, and the fraction of executed actions whose successor is non-viable.

We compute net inference cost as

$$\text{NetCompute} = \frac{\text{PolicyTokens} + K \cdot \text{EvalTokens}}{\text{PolicyTokens}_{\text{top-1}}}. \tag{12}$$

This prevents a misleading comparison in which a method appears better only because it spends more inference computation per decision.

3.6 Candidate-Exhaustion Termination

Finally, we evaluate the termination branch of Algorithm 2. This rule should be interpreted conservatively. It does not prove that the current state is globally unsolvable. It only states that, under the sampled candidate budget K, none of the proposed successors exceeds the fallback viability threshold:

$$\max_{a \in A_K(s)} \hat{v}_\phi(s, a) < \tau_{\text{fb}}.$$

We call this *candidate-exhaustion termination*.

Table 4: Online rollout performance under matched candidate budget. Net compute includes candidate evaluation overhead. Bold marks the best deployable method, excluding oracle viability.

Method	Pass@1 ↑	DE entry ↓	First DE step ↑	Non-viable ↓	Tokens/ep ↓	Net compute ↓
Policy top-1	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	1.00
Best-of-K policy	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Local progress heuristic	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Prompted score-only	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Learned progress-score	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
SAVE	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	[TBD]	[TBD]
Oracle viability	[TBD]	0.00	–	0.00	[TBD]	[TBD]

Table 5: Candidate-exhaustion termination. False termination is the fraction of terminated states that were still recoverable according to the oracle solver. Tokens saved is measured relative to SAVE without termination.

Method	False term. ↓	True doomed term. ↑	Tokens saved ↑	Pass@1 drop ↓
No termination	0.00	–	0.0	0.00
Greedy termination	[TBD]	[TBD]	[TBD]	[TBD]
SAVE termination	[TBD]	[TBD]	[TBD]	[TBD]
SAVE termination + retry	[TBD]	[TBD]	[TBD]	[TBD]
Random matched-rate	[TBD]	[TBD]	[TBD]	[TBD]
Oracle termination	0.00	100.0	[TBD]	0.00

The main risk is false termination: stopping at a state from which a successful continuation still exists. We therefore report false termination rate as the primary safety metric, and tokens saved as the primary efficiency metric.

The retry variant resamples one additional candidate set before terminating. It controls for the possibility that the first candidate set was poor even though the state itself remained recoverable. We include it to separate model uncertainty from finite-sample candidate exhaustion.

3.7 Ablations

We ablate the components that make SAVE action-conditioned and deployable. The main table reports results on [Pentomino / strongest environment / TBD], with full results for all environments in Appendix **??**.

These ablations distinguish three effects. Removing sibling supervision tests whether the model needs same-state alternatives rather than only executed actions. Removing contrastive ranking tests whether the pairwise loss improves discrimination near the viability boundary. Removing calibration or collapsing the two thresholds tests whether the deployment rule depends on calibrated probabilities rather than uncalibrated logits.

3.8 Summary

Together, the experiments test SAVE at three levels: prediction, counterfactual candidate choice, and online behavior. The prediction results measure whether viability is learned and calibrated. The deceptive benchmark tests whether viability differs from progress or utility scoring. The rollout and termination experiments test whether calibrated viability changes the agent's executed trajectory under a fixed candidate budget.

References

References follow the acknowledgments in the camera-ready paper. Use unnumbered first-level heading for the references. Any choice of citation style is acceptable as long as you are consistent. It is permissible to reduce the font size to `small` (9 point) when listing the references. Note that the Reference section does not count towards the page limit.

[Placeholder for τ_{fb} sweep.]

Figure 4: Trade-off induced by the fallback threshold τ_{fb}. Lower thresholds reduce false termination but save less compute; higher thresholds save more compute but may terminate recoverable trajectories.

Table 6: Ablation study. Same-state pairwise accuracy measures action-conditioned discrimination; non-viable selected measures the operational effect during rollout.

Variant	Viab. AUC ↑	Same-state pairwise ↑	Pass@1 ↑	Non-viable selected ↓
SAVE	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
Executed actions only	[TBD]	[TBD]	[TBD]	[TBD]
No contrastive ranking	[TBD]	[TBD]	[TBD]	[TBD]
No boundary-aware sampling	[TBD]	[TBD]	[TBD]	[TBD]
State-conditioned only	[TBD]	[TBD]	[TBD]	[TBD]
No calibration	[TBD]	[TBD]	[TBD]	[TBD]
Single threshold	[TBD]	[TBD]	[TBD]	[TBD]
$K = 1$	[TBD]	–	[TBD]	[TBD]
$K = 2$	[TBD]	[TBD]	[TBD]	[TBD]
$K = 4$	[TBD]	[TBD]	[TBD]	[TBD]
$K = 8$	[TBD]	[TBD]	[TBD]	[TBD]

References

Shiqi Chen, Tongyao Zhu, Zian Wang, Jinghan Zhang, Kangrui Wang, Siyang Gao, Teng Xiao, Yee Whye Teh, Junxian He, and Manling Li. Internalizing world models via self-play finetuning for agentic rl. *arXiv preprint arXiv:2510.15047*, 2025.

Yang Chu et al. The road not taken: Hindsight exploration for LLMs in multi-turn RL. In *International Conference on Machine Learning (ICML)*, 2025. Author list to verify from official proceedings.

Kangrui Wang, Pingyue Zhang, Zihan Wang, Yaning Gao, Linjie Li, Qineng Wang, Hanyang Chen, Chi Wan, Yiping Lu, Zhengyuan Yang, Lijuan Wang, Ranjay Krishna, Jiajun Wu, Li Fei-Fei, Yejin Choi, and Manling Li. VAGEN: Reinforcing world model reasoning for multi-turn VLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

A Output Schema for SAVE Heads

[To be added.]

B Sibling Sampling Protocol

[To be added.]

C Perturbed Solver-Path Generators

[To be added.]

D Hyperparameters and Calibration Protocol

[To be added.]